

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D5 — Retrospective

Part 1 Submission

8 May 2026

Halit Batuhan ASLAN

220104004003

Contents

Retrospective

Retrospective;

What is working. The lexer and parser are fully functional and consistent with the EBNF grammar in D1. The lexer correctly tokenizes all “ayar” constructs — keywords, five literal types (INT, FLOAT, STRING, BOOL, METRIC), all operators including two-character forms, and separators — with accurate line and column tracking. The parser implements a clean recursive-descent architecture with one method per EBNF non-terminal, correctly encoding operator precedence through stratified nonterminals and left-associativity through iterative while-loops. The layout validation pass enforces the curly-brace-on-its-own-line rule as a pre-parse step. All 5 valid sample programs parse successfully and produce correct ASTs; all 7 invalid programs trigger informative ParseError or LayoutError messages with exact source locations. The test suite (34 tests) passes at 100%.

What is hard. The most challenging aspect so far has been maintaining consistency between the EBNF grammar, the lexer, and the parser as the design evolved. The analyze block’s separator convention (semicolon-terminated fields with no comma between them) diverged from the model block’s convention (comma-separated fields), and this inconsistency caused multiple rounds of corrections across all three artifacts. Every time I changed a grammar rule, the test suite broke and had to be regenerated. Another difficulty was integrating AI-generated code with my own: the AI consistently assumed different token type names than my lexer actually produces, requiring manual correction of 8 mismatched identifiers in the parser. Looking ahead, the type checker and interpreter for Part 2 will be significantly harder because they require semantic analysis — enforcing name equivalence for the metric type, detecting data leakage (train on test set), and simulating ML training with deterministic metric generation.

What I plan to fix in Part 2. First, I will add the type checker to enforce strong typing: metric values must not be used in arithmetic, int→float coercion applies only in mixed-mode arithmetic, and experiment records use name equivalence. Second, I will implement the interpreter with static scoping, where model declarations live in global scope and experiment bodies create their own inner scope. The interpreter must simulate training and generate deterministic metrics, then execute the compare/exclude/select pipeline to choose the best model. Third, I will add runtime error detection for domain-specific violations: training on the test split (data leakage), division by zero, and referencing undefined models. Finally, I plan to revise the D1 design specification to add the sections required for Part 2 (semantics in operational style for the experiment block and the compare...select construct, the full type system description, and the design rationale with Sebesta terminology). I also want to clean up the analyze block’s separator convention and document it explicitly in D1 to avoid future confusion.